in D if $s_i = s_j$. Given a dataset D, find all duplicates.

Example: $D = (A, C, B, Z, C, B, C)$ (positions 1 2 3 4 5 6 7), set of duplicates in D: $Dupl(D) = \{(2,5), (2,7), (5,7), (3,6)\}$

Simpler algorithm: Sort $((s_1,1), \dots, (s_n,n))$ by their first coordinate, iterate over the sorted sequence to find duplicates

Example:

Indexing:	(A,1)	(C,2)	(B,3)	(Z,4)	(C,5)	(B,6)	(C,7)
Sorting:	(A,1)	(B,3)	(B,6)	(C,2)	(C,5)	(C,7)	(Z,4)
Duplicates:		(3,6)		(2,5), (2,7), (5,7)			

Analysis: Sorting takes $O(n \log n)$ comparisons and memory access, iterating takes $O(n + |Dupl(D)|)$

→ **Reason for Hashmaps:** It is more efficient to compare hash values than large objects

→ Let the elements in D be from a universe U. We use a hash function $h: U \to [m]$ and require:

→ h is efficiently computable

→ h acts as a randomized function, meaning $\forall u \in U \; \forall i \in [m]: Pr[h(u) = i] = \frac{1}{m}$

→ Important: Each $h(s_i)$ is randomized over $[m]$, but $s_i = s_j \Rightarrow h(s_i) = h(s_j)$, we compress by choosing m much smaller than U

Example solution:

Input:	A	C	B	Z	C	B	C
Hash and index:	(31,1) h(A)	(27,2) h(C)	(12,3) h(B)	(12,4) h(Z)	(27,5) h(C)	(12,6) h(B)	(27,7) h(C)
Sort:	(12,3)	(12,4)	(12,6)	(27,2)	(27,5)	(27,7)	(31,1)
Duplicates:		(3,6),		(2,5), (2,7), (5,7)			

→ Important: check for collisions while finding duplicates, such as $h(B) = h(Z)$

→ Collisions are the new, unwanted duplicates in a hashmap $\Rightarrow$ pairs (i,j), $1 \le i < j \le n$ with $s_i \neq s_j$ and $h(s_i) = h(s_j)$

Probability of a collision: Let K_{ij} be the indic. var. of „(i,j) is a collision". Then:

$Pr[K_{ij} = 1] = \begin{cases} \frac{1}{m} & \text{if } s_i \neq s_j \\ 0 & \text{if } s_i = s_j \end{cases} \Rightarrow \mathbb{E}[K_{ij}] \le \frac{1}{m} \; \forall i,j \Rightarrow \mathbb{E}[\#\text{collisions}] \le \sum_{1 \le i < j \le n} \underbrace{\mathbb{E}[K_{ij}]}_{\le \frac{1}{m}} \le \underbrace{\binom{n}{2}}_{\text{pairs with } i<j \text{ are half of total}} \cdot \frac{1}{m}$

$\Rightarrow$ for $m = n^2$, $\mathbb{E}[\#\text{collisions}] < 1$, which only adds a constant factor to the runtime

Runtime: n hash function computations, $O(n \log n)$ to sort $\lceil 2 \log n \rceil$-bit integers, $|Dupl(D)| + \#\text{collisions}$

$\Rightarrow$ same asymptotic runtime as simple algorithm but the comparisons themselves are less expensive for more complex objects

General Algorithms: (W10-W13 starts here)

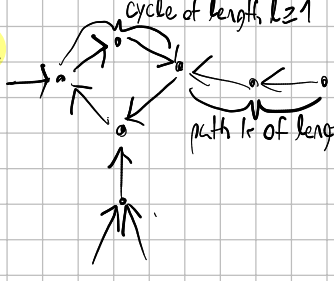
Floyd's cycle finding alg. (Hare & Tortoise):

Given an array $a[1, \dots, n]$ with $a[i] \in \{1, \dots, n-1\}$ (n-1 unique values), find indices $i \neq j$ with $a[i] = a[j]$ in $O(n)$ time, $O(1)$ space: doing one of each is easy: optimize space by remembering previous values in $O(n^2)$ time or use BucketSort for $O(n)$ time and space.

→ Consider the directed graph $D = (V, A)$ with $V = [n]$, $A = \{(i, a[i]) \mid 1 \le i \le n\}$

→ how does D look? each vertex has one outgoing and one ingoing edge → the subgraph D_n consists of vertices reachable from n

→ **Example:**



→ consider the sequence of vertices $x_0 = n, x_t := a[x_{t-1}] \; \forall t \ge 1$

→ Claim: $\exists t \le n$ s.t. $x_t = x_{2t}$

→ **Proof (optional):** Let $r = \lceil \frac{t}{k} \rceil \cdot k - t \ge 0$. Then x_{t+r} is in the cycle C.

$\Rightarrow t + r = \lceil \frac{t}{k} \rceil \cdot k$ is an integer multiple of k (length of C) $\Rightarrow x_{t+r} = x_{2(t+r)}$

→ Additionally, $f := k+r = \lceil \frac{n}{\ell} \rceil \cdot \ell < (\frac{n}{\ell}+1)\cdot \ell = k+\ell \leq n+1$.
↳ D_a has $k+\ell$ vertices

→ **Claim:** Let $t \leq h$ with $x_t = x_{2t}$, then: $x_{t+k} = x_k$ ⇒ **Proof:** $x_t = x_{2t} \Rightarrow 2t = t+p\cdot\ell$ for $p \in \mathbb{N} \Rightarrow x_{t+t} = x_t$

Hare & Tortoise Algorithm:

hare → a[a[n]] (2 steps in graph), tortoise → a[n] (1 step in graph), t → 1

```
while (hare ≠ tortoise)
    tortoise → a[tortoise]
    hare → a[a[hare]]
    t → t+1
```
→ cycle-finding portion

↳ 3 variables in memory

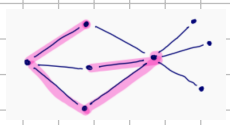
hare → n

```
while (hare ≠ tortoise)
    i → hare
    j → tortoise
    tortoise → a[tortoise]
    hare → a[hare]
```
2 additional vars. in mem. → locate i and j with a[i] = a[j]

Long Path Problem:

→ Given a graph $G=(V,E)$ and $\beta \in \mathbb{N}_0$, determine if there exists a path of length β in G.

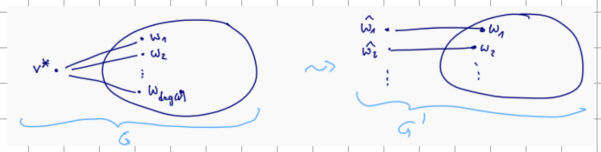
Example:



→ yes for $\beta \leq 4$, remember on a path we cannot repeat vertices

Theorem: If we can decide long-path for graphs with n vertices in time $t(n)$, we can decide if a graph with n vertices has a Hamiltonian cycle in time $t(2n-2)+O(n^2)$. (recall that finding a Hamiltonian cycle is NP-complete)

Proof (optional): We have to show that if $G(V,E)$ has Hamiltonian cycle $\Leftrightarrow$ G' has a path of length n, with G' being a graph we can construct in time $O(n^2)$ with $n' \leq 2n-2$ vertices

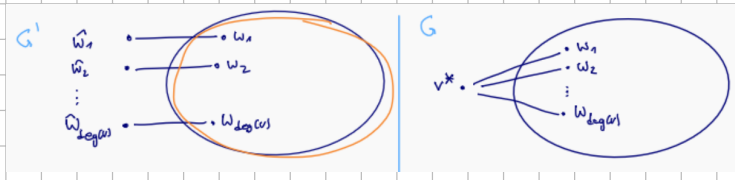


→ Let v^* be any vertex. Remove v^*. Replace the edges $\{v^*, w_1\}, \dots, \{v^*, w_{deg(v^*)}\}$ with new edges $\{w_1, \hat{w}_1\}, \dots, \{w_{deg(v^*)}, \hat{w}_{deg(v^*)}\}$ with $\hat{w}_1, \dots, \hat{w}_{deg(v^*)}$ being new edges.

→ Show that G has Hamiltonian cycle ⇒ G' has a path of length n:

→ Let $\langle v_1, \dots, v_n, v_1 \rangle$ be a Hamiltonian cycle in G.

→ Assume that $v_1 = v^*$. Then, $\langle \hat{v}_2, \dots, v_n, \hat{v}_n \rangle$ is a path of length n in G'.



Now show the opposite direction:

→ Let $\langle u_0, \dots, u_n \rangle$ be a path of length n in G'.

→ Vertices $u_1, \dots, u_{n-1}$ have degree ≥ 2 and are pairwise distinct

⇒ These are the vertices in $V \setminus \{v^*\}$. Therefore, let $u_0 = \hat{w}_i$ and $u_n = \hat{w}_j$ be pairwise distinct new vertices with degree 1 in G'.

⇒ $\langle v^*, u_1, \dots, u_{n-1}, v^* \rangle$ is a Hamiltonian cycle in G.

Goal → solve the long path problem in $O(c^\beta)$ for a constant c, brute force takes $O(n^{\beta+2})$

Color coding:

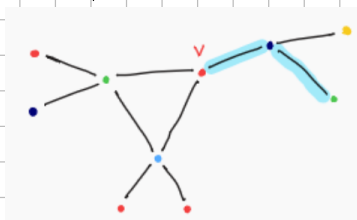
Idea: Color vertices randomly, find "colorful paths" (bunte Pfade)

→ Let $G=(V,E)$ be a graph with a coloring $\gamma: V \to [k]$. A path is colorful if all vertices have different colors $\gamma(v)$.

Checking for colorful paths: Given a graph $G=(V,E)$ and a coloring $\gamma: V \to [k]$, decide if there exists a colorful path of length $k-1$.

→ $\forall v \in V$ and $i \in \{0,\dots,k-1\}$ let $P_i(v) := \{S \subseteq [k] : |S| = i+1$ and $\exists$ a colorful path colored in S that ends with $v\}$

Example:



$P_2(v) = \{\{\cdot,\cdot,\cdot\}, \{\cdot,\cdot,\cdot\}, \{\cdot,\cdot,\cdot\}\}$

Observation: $\exists$ colorful path of length $k-1 \iff \bigcup_{v \in V} P_{k-1}(v) \neq \emptyset$

Recursive formula for $P_i(v)$: $P_0(v) = \{\{\gamma(v)\}\}$

For $i \geq 1$: $\exists$ a colorful path colored with S that ends in v

$\iff \exists$ neighbor x of v and a colorful path colored with $S \setminus \{\gamma(v)\}$ that ends in x

→ Therefore, $P_i(v) = \bigcup_{x \in N(v)} \{R \cup \{\gamma(v)\} \mid R \in P_{i-1}(x)$ and $\gamma(v) \notin R\}$

Algorithm Colorful(G,i), which computes $P_i(v)$ $\forall v \in V$ given $P_{i-1}(v)$ $\forall v \in V$:

```
1: for all v ∈ V do
2:     P_i(v) ← ∅
3:     for all x ∈ N(v) do
4:         for all R ∈ P_{i-1}(x) mit γ(v) ∉ R do
5:             P_i(v) ← P_i(v) ∪ {R ∪ {γ(v)}}
```

Runtime analysis:

→ # of subsets of $[k]$ with i elements: $\binom{k}{i} \Rightarrow |P_{i-1}(x)| \leq \binom{k}{i}$

→ Runtime of colorful(G,i): $O(\sum_{v \in V} \deg(v) \binom{k}{i} \cdot i) = O(\binom{k}{i} \cdot i \cdot m)$ ($m = |E| = \frac{1}{2}\sum_{v \in V} \deg(v)$; i ← size of sets in $P_{i-1}(v)$)

→ Total runtime (running alg. from 1 until k): $\sum_{i=1}^{k-1} \binom{k}{i} i \cdot m \leq \sum_{i=0}^{k} \binom{k}{i} k \cdot m = 2^k \cdot k \cdot m \Rightarrow$ Runtime $= O(2^k \cdot k \cdot m)$

Random colorings:

→ Assume G has a path P of length $k-1$. $\Rightarrow \exists k^k$ colorings of P with k colors, and $k!$ colorful colorings of P with k colors

$\Rightarrow$ If we randomly color the vertices of G, $Pr[\exists$ colorful path$^{(*)}$ of length $k-1] \geq Pr[P$ is colorful$] \geq \frac{k!}{k^k} \geq e^{-k}$

($\geq$ power series e^x: $e^k = \sum_{i=0}^{\infty} \frac{k^i}{i!} \geq \frac{k^k}{k!}$)

Theorem: Let G be a graph with a path of length $k-1$.

1) A random coloring with k colors generates a colorful path of length $k-1$ with probability $p \geq e^{-k}$

2) For repeated colorings with k colors, the expected value of attempts until there is a colorful path of length $k-1$ is $\frac{1}{p} \leq e^k$

($\leftarrow$ in Geo(p) $\Rightarrow E = \frac{1}{p}$)

Monte Carlo algorithm for long path:

① Repeat the following $\lceil \lambda e^k \rceil$ times:

→ color the vertices of G randomly and uniformly with $k = (3+1)$ colors and verify if $\exists$ a colorful path of length $k-1$

→ if yes, return YES

② If no iteration returned YES, return NO

→ Runtime: $O(\lambda \cdot e^k \cdot 2^k \cdot k \cdot m) = O(\lambda \cdot (2e)^k \cdot k \cdot m)$

Theorem: 1) The algorithm has runtime $O(\lambda (2e)^k km)$

2) If the algorithm returns YES, the graph has a path of length $k-1$

3) If the graph has a path of length $k-1$, the probability of the algorithm returning NO is maximally $e^{-\lambda}$

Proof of 3): $Pr[NO] \leq (1-e^{-k})^{\lceil \lambda e^k \rceil} \leq \left(e^{e^{-k}}\right)^{\lceil \lambda e^k \rceil} = e^{-e^k \cdot \lceil \lambda e^k \rceil} \leq e^{-\lambda}$

($\leftarrow 1+x \leq e^x \ \forall x \in \mathbb{R}$, here $x = e^{-k}$) ∎

→ the algorithm only runs in polynomial time for paths of length $\leq \log(n)$

→ deterministic version: **Theorem:** $\exists$ a family F of colorings $\gamma: V \to [2k]$ s.t.:

→ $\forall A \subseteq V$ with $|A| \leq k$: there is a coloring in F s.t. A is colorful with coloring γ

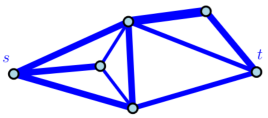
→ $|F|$ has polynomial size, meaning it can be constructed in polynomial time

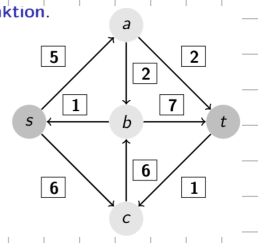
$k = \mathcal{O}(\log n)$

Flow in networks:

Def.: A network is a tuple $N = (V, A, c, s, t)$ with (V, A) being a directed acyclic graph (DAG), $s \in V$ being the source, $t \in V \setminus \{s\}$ being the sink, and $c: A \to \mathbb{R}_0^+$ being the capacity function

► $c : A \to \mathbb{R}_0^+$, die Kapazitätsfunktion.

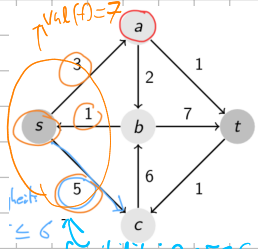




Def.: Let $N = (V, A, c, s, t)$ be a network. A flow in N is a function $f: A \to \mathbb{R}$ with the following conditions:

Feasibility: $0 \leq f(e) \leq c(e)$ $\forall e \in A$, **flow conservation:** $\forall v \in V \setminus \{s,t\}: \sum_{\substack{u \in V: \\ (u,v) \in A}} f(u,v) = \sum_{\substack{u \in V: \\ (v,u) \in A}} f(v,u)$

Value of a flow: $\text{val}(f) = \text{netoutflow}(s) := \underbrace{\sum_{v \in V: (s,v) \in A} f(s,v)}_{\text{outflow}} - \underbrace{\sum_{v \in V: (v,s) \in A} f(v,s)}_{\text{inflow}}$



Lemma: The net inflow of the sink t equals the value of the flow:

$\text{netinflow}(t) := \sum_{v \in V: (v,t) \in A} f(v,t) - \sum_{v \in V: (t,v) \in A} f(t,v) = \text{val}(f)$

Proof: $0 = \sum_{(v,u) \in A} f(v,u) - \sum_{(u,v) \in A} f(u,v) = \sum_{v \in V} \Big(\underbrace{\sum_{\substack{u \in V: \\ (v,u) \in A}} f(v,u) - \sum_{\substack{u \in V: \\ (u,v) \in A}} f(u,v)}_{0 \text{ for } v \neq s,t \text{ (flow conservation)}} \Big) = \Big(\underbrace{\sum_{\substack{u \in V: \\ (s,u) \in A}} f(s,u) - \sum_{\substack{u \in V: \\ (u,s) \in A}} f(u,s)}_{= \text{val}(f)} \Big)$

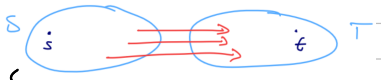
$+ \Big(\underbrace{\sum_{\substack{v \in V: \\ (t,v) \in A}} f(t,v)}_{= \text{netinflow}(t)} - \sum_{\substack{v \in V: \\ (v,t) \in A}} f(v,t) \Big)$ ∎

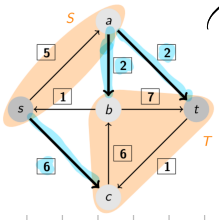
Maxflow Problem: Given a network $N = (V, A, c, s, t)$, find a flow f with the highest value (a max. flow)

→ Does this always exist? How would you know if a flow is maximal?

Def.: An s-t cut for a network (V, A, c, s, t) is a partition (S, T) of V with $s \in S$ and $t \in T$. ($S \cup T = V$, $S \cap T = \emptyset$)

The capacity of an s-t cut (S, T) is $\text{cap}(S, T) := \sum_{(v,w) \in (S \times T) \cap A} c(v,w)$ ← edges from S to T, not T to S



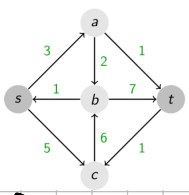


↷ $\text{cap}(S, T) = 6 + 2 + 2 = 10$

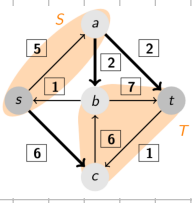
Lemma: Let f be a flow and (S, T) an s-t cut. $\text{val}(f) \leq \text{cap}(S, T)$ (I will omit the proof)

Consequence: If for a flow f, we find an s-t cut (S, T) with $\text{cap}(S, T) = \text{val}(f)$, f is a maximal flow.

⇒ (S, T) is a simple certificate for the maximality of f.



Flow with $\text{val}(f) = 7$



s-t cut with $\text{cap}(S, T) = 10$

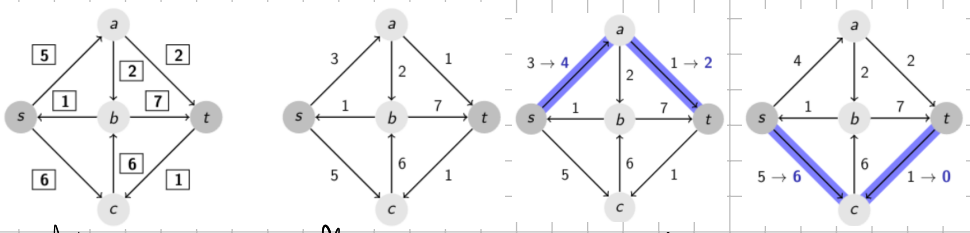
Maxflow-Mincut Theorem: For any network $N = (V, A, c, s, t)$,

$\max_{\text{flow } f} \text{val}(f) = \min_{s\text{-}t \text{ cut } (S,T)} \text{cap}(S, T)$

⇒ s-t-mincut is a finite algorithmic problem and a min. cut always exists (there is a finite amount of s-t cuts)

Goal: Algorithm and proof for the aforementioned theorem

Improving a flow:

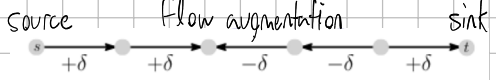


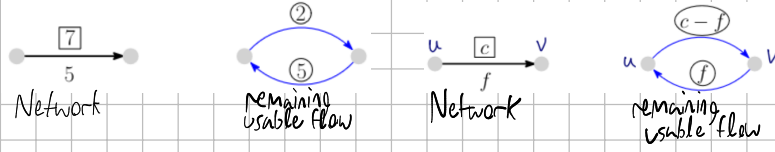
Network N | Flow f, val(f)=7 | val(f)=8 | val(f)=9

We can change the flow while maintaining flow conservation in the following ways:

→ considering: { the capacity for "+δ" / the current flow for "−δ"

Finding augmenting paths / handling remaining flow:

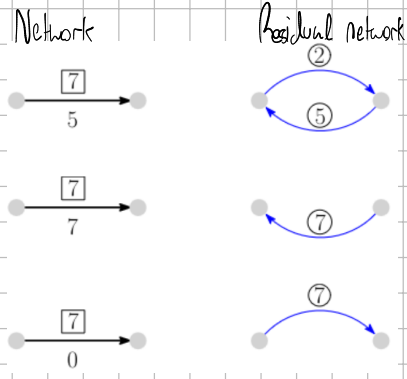




→ Notation: For $e=(v,v)$, let $e^{opp} := (v,v)$

Def.: Let $N=(V,A,c,s,t)$ be a network (without opposite edges) and f be a flow in N. The residual network $N_f := (V, A_f, r_f, s, t)$ is defined as:

1) If $e \in A$ with $f(e) < c(e)$, e is an edge in A_f with $r_f(e) := c(e) - f(e)$
2) If $e \in A$ with $f(e) > 0$, e^{opp} is an edge in A_f with $r_f(e^{opp}) = f(e)$
3) A_f only contains edges as outlined in 1) and 2)



→ The value $r_f(e)$ is known as the remaining capacity of an edge e

Theorem: Let N be a network (without opposite edges).

A flow f is maximal if and only if the residual network N_f has no directed s-t path.

Each maximal flow f has an s-t cut (S,T) with $val(f) = cap(S,T)$

Proof: Claim: The residual network N_f has a directed s-t path $\Rightarrow$ f can be augmented ($\Leftrightarrow$ f not maximal)

Proof: Consider a directed s-t path in N_f:  and take the smallest residual capacity $\varepsilon := \min\{\varepsilon_i\}$, then augment f along the path around ε

Claim: The residual network N_f has no directed s-t path $\Rightarrow$ $\exists$ s-t cut (S,T) with $cap(S,T) = val(f)$ ($\Leftrightarrow$ f maximal)

Proof: $S :=$ vertices reachable from S in N_f, $T := V \setminus S$ $\Rightarrow$ $s \in S, t \notin S$ (no directed s-t path in N_f) $\Rightarrow$ (S,T) is s-t cut

Claim: $cap(S,T) = val(f)$ (using S and T we defined in last proof, recall $(S \times T) \cap A_f = \emptyset$)

Proof: $\forall e \in (S \times T) \cap A: f(e) = c(e) \Rightarrow f(S,T) = cap(S,T)$
$\forall e' \in (T \times S) \cap A: f(e') = 0 \Rightarrow f(T,S) = 0$
$\Rightarrow val(f) = \overbrace{f(S,T)}^{= cap(S,T)} - \overbrace{f(T,S)}^{=0} = cap(S,T)$ ∎

Ford-Fulkerson Algorithm:

1: $f \to 0$ (flow constant 0) → we cannot guarantee that the algorithm terminates

2: while $\exists$ s-t path P in N_f do (augmenting path) → the algorithm may run infinitely for capacities in $\mathbb{R}$

3: Augment flow along P → for capacities in $\mathbb{N}_0$, the flows and remaining capacities stay integers, and the flow value is increased by 1 each augmenting step

4: return f (max. flow)

Runtime analysis: Let $n := |V|$ and $m := |A|$ for a network $N = (V, A, c, s, t)$

→ Assume $c: A \to \mathbb{N}_0$ and $U := \max_{e \in A} c(e)$. Then, $val(f) \leq cap(\{s\}, V \setminus \{s\}) \leq (n-1)U$, meaning there are at most $(n-1)U$ augmenting steps

→ One augmenting step (search for s-t path in N_f, augment, update N_f) is done in $O(m)$ $\Rightarrow$ total runtime $O(m \cdot n \cdot U)$

Theorem (Ford-Fulkerson with integer capacities): Let $N = (V, A, c, s, t)$ be a network with $c: A \to \mathbb{N}_0^{\leq U}$ (for $U \in \mathbb{N}$) (without opposite edges). Then, there is an integer maximal flow that can be computed in $O(mnU)$

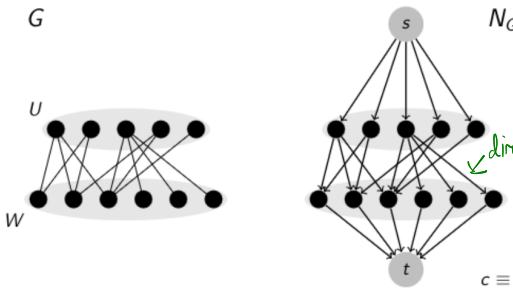
→ That concludes our proof of the Maxflow-Mincut Theorem, I think this week's peer graded exercise (P5) is great practice

Practical uses of flow:

Max. Bipartite Matching Problem: Find a maximal matching for an unweighted, undirected graph:

→ Matchings are covered on page 5 if you want a recap

→ The greedy algorithm won't be maximal → we solve this by reducing it to a maxflow problem

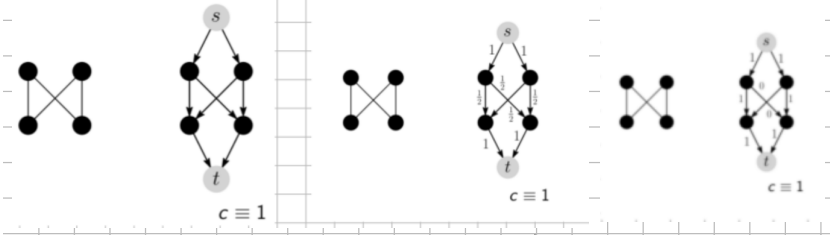


→ We map each bipartite graph with given vertex partition $U \dot\cup W$ onto a network:

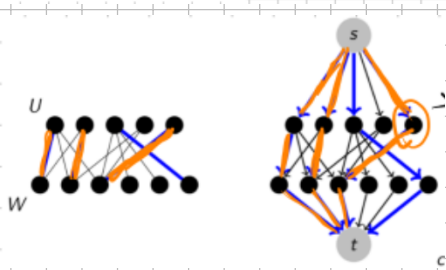
$G = (U \dot\cup W, E) \mapsto N_G = (U \dot\cup W \dot\cup \{s,t\}, A, c, s, t)$ (bipartite graph; network; edges)

→ $s \neq t$ are added vertices

→ $A := \{s\} \times U \cup \{(u,w) \in U \times W \mid \{u,w\} \in E\} \cup W \times \{t\}$ → $c \equiv 1$.



→ By the Theorem for Ford-Fulkerson, there is an integer maxflow computable in $O(mnU)$ with $U \equiv 1$ here



→ A matching M in G maps to an integer flow f_M in N_G with $val(f_M) = |M|$

→ either 0 flows in and 0 flows out or 1 flows in and out of a vertex → Matching conditions: we cannot choose 2 neighboring vertices $\Rightarrow$ the integer flow f in N_G also maps to the matching

$\Rightarrow$ max. matching in G $\;=\;$ "integer maxflow in N_G"

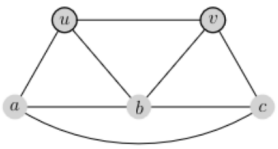
→ $\max_{\text{matching in } G} |M| = \max_{f \text{ integer flow in } N_G} val(f) = \max_{f \text{ flow in } N_G} val(f)$

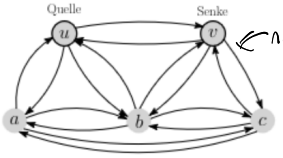
Edge-disjoint paths:

Problem: Given G with 2 selected edges u, v, $u \neq v$, compute the max. set of edge-disjoint u-v paths

→ Recap on Menger on page 1

→ We map $\hat{G} = (V, E), u, v \in V \mapsto N_G^* = (V, A, c, u, v)$, $A := \{(x,y), (y,x) \mid \{x,y\} \in E\}$, $c \equiv 1$ (graph with 2 vertices; network)



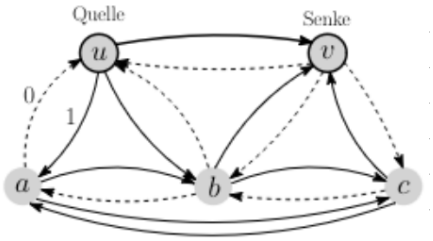


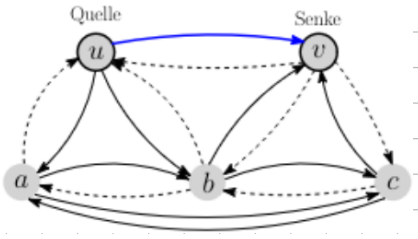
← notice how opposite edges are allowed

→ Compute an integer maxflow f in N_G^* ⇒ flow values $\in \{0,1\}$

→ $\forall w \notin \{u,v\}$: $\text{indeg}_f(w) = \text{outdeg}_f(w)$

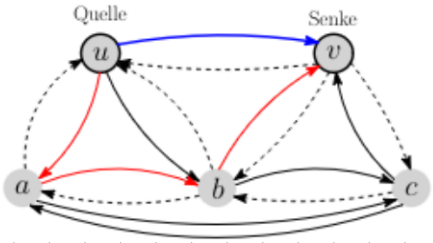
→ $\text{val}(f) = \text{outdeg}_f(v) - \text{indeg}_f(v) = \text{indeg}_f(v) - \text{outdeg}_f(v)$ (in/out degrees concerning edges with flow = 1)

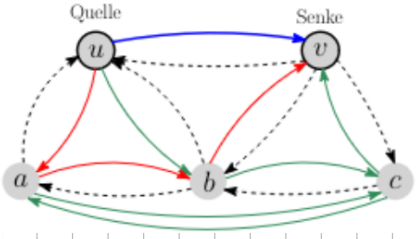


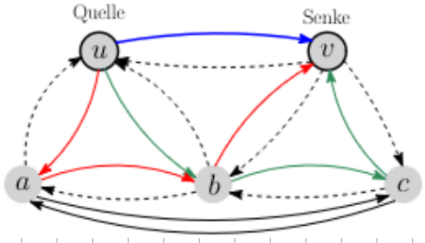


→ starting at v, go along unused edges with flow = 1 until v is reached. any edge used along the way is marked as such.

→ repeat $\text{val}(f)$ times. we get $\text{val}(f)$ edge-disj. paths







We just proved **Menger's Theorem** in a roundabout way: Let G be a graph with vertices u and v, $u \neq v$:

max # edge-disjoint paths in G = maxflow(N_G) = min cap(S,T) for u-v-cut (S,T) in N_G = min #edges separating u and v

Mincut Problem:

Def.: A **multigraph** is an undirected, unweighted graph $G = (V, E)$ without loops, but allowing for multiple edges between a single pair of vertices (we can also see this as integer weights)

Def.: An **edge cut** in a multigraph $G = (V, E)$ is a set of vertices C so that $(V, E \setminus C)$ is not connected.

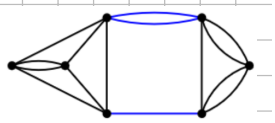
Def.: The **cardinality of a minimum cut** in G, $\mu(G)$, is defined as $\mu(G) := \min_{C \subseteq E,\ (V, E\setminus C) \text{ not connected}} |C|$

(Equivalently, we can see a cut as a partition $V = S \dot\cup T$ with $S \neq \emptyset$, $T \neq \emptyset$, and minimize the edges between S and T)

Mincut problem: Given a multigraph G, compute $\mu(G)$.

→ for G that is not connected, $\mu(G) = 0$

$\mu = 4$
min deg $= 4$

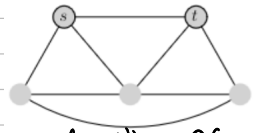


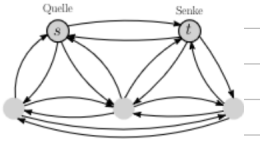
$\mu = 3$, min deg $= 4$

Def.: For a multigraph $G = (V, E)$, the **degree** $\deg(v) = \deg_G(v)$ of a vertex is the amount of **incident edges** (not neighbors), therefore: $|E| = \frac{1}{2}\sum_{v \in V} \deg(v)$ and $\mu(G) \leq \min_{v \in V} \deg(v)$.

Naive approach:

→ we can already compute the smallest s-t cut → smallest amount of edges to remove to separate s from t → this is done in $O(mnU) = O(mn(1 + \log U))$ or $O(mn \log n)$



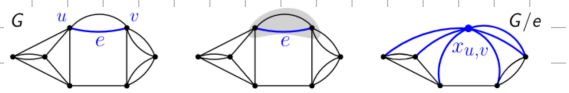


→ Now, fix one s and consider all $t \in V \setminus \{s\}$ → every cut is an s-t cut for some $t \in V \setminus \{s\}$

→ repeat $O(mn \log n)$ $(n-1)$ times ⇒ $O(mn^2 \log n) = O(n^4 \log n)$

Observation: the lecture also used an em-dash

Next approach ≙ edge contraction: Given: $G=(V,E)$, $e=\{u,v\}\in E$:



Contraction of e: melts u,v to a new edge $x_{u,v}$ that is incident to all edges that were incident to u or v, edges between u and v disappear.

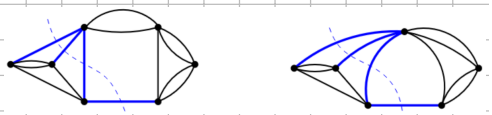
→ We call the new graph G/e. For $k:=$ #edges between u and v: $\deg_{G/e}(x_{u,v}) = \deg_G(u)+\deg_G(v)-2k$ and $|E(G/e)| = |E(G)|-k$

→ Let $e=\{u,v\}$. ∃ a natural bijection:

$E(G)$ without edges between u and v → $E(G/e)$.

→ For $w,w' \in V(G)\setminus\{u,v\}$: $\{w,w'\}\mapsto\{w,w'\}$, $\{w,u\}\mapsto\{w,x_{u,v}\}$, $\{w,v\}\mapsto\{w,x_{u,v}\}$

→ This brings about a bijection: Cuts in G without e → all cuts in G/e



Lemma: Let $G=(V,E)$ be a multigraph and $e\in E$. Then $\mu(G/e)\geq\mu(G)$.

If G has a mincut C with $e\notin C$, $\mu(G/e)=\mu(G)$.

→ μ can never fall by contracting some edge e, μ stays the same if there exists a mincut without e

How do we find an edge e that contains μ?

Algorithm:

CUT(G) — G connected multigraph

1: $G' \leftarrow G$
2: **while** $|V(G')|>2$ **do**
3: $e \leftarrow$ uniformly distributed random edge in G'
4: $G' \leftarrow G'/e$
5: **return** size of unique cut in G'

→ Let $n:=|V(G)|$. Prerequisites: Contraction and random selection in $O(n)$

→ (This requires viewing multiple edges as edge weights) → CUT(G) runs in $O(n^2)$

→ how does the computed value help us?

→ The algorithm never returns a value $<\mu(G)$

→ If there is a minimal cut C from which no edge is contracted, the algorithm returns $\mu(G)$.

Lemma: Let $G=(V,E)$, $n:=|V|$. For an edge e that is randomly selected from the edges in G, $Pr[\mu(G)=\mu(G/e)]\geq 1-\frac{2}{n}$.

Def.: Success probability $\hat{p}(G)$ of CUT(G) returning $\mu(G)$: $\hat{p}(n):=\inf_{G=(V,E),|V|=n}\hat{p}(G)$

Lemma: $\forall n\geq 3$: $\hat{p}(n)\geq\left(1-\frac{2}{n}\right)\cdot\hat{p}(n-1)$

Lemma: $\forall n\geq 2$: $\hat{p}(n)\geq\frac{2}{n(n-1)}=1/\binom{n}{2}$ ⇒ expected value of repetitions until we first return $\mu(G)$ is at most $\binom{n}{2}$.

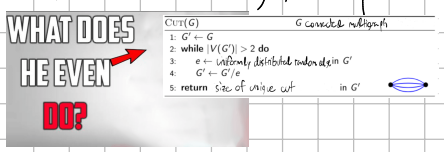
Idea: We repeat CUT(G) $\lambda\binom{n}{2}$ times for some $\lambda>0$ and return the smallest value ever gotten

Theorem: For the algorithm consisting of $\lambda\binom{n}{2}$-time repetition of CUT(G):

1) The algorithm has a runtime of $O(\lambda n^4)$. 2) The smallest encountered value is equal to $\mu(G)$ with a probability of at least $1-e^{-\lambda}$.

→ With $\lambda:=\ln(n)$, we have time $O(n^4\log n)$ with probability of error $\leq 1/n$.

→ one has to ask:



and actually, we can improve it using bootstrapping:

$$\hat{p}(n) \geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdots \underbrace{\frac{3}{5}}_{0.6} \cdot \underbrace{\frac{2}{4}}_{0.5} \cdot \underbrace{\frac{1}{3}}_{0.33} \cdot \underbrace{\hat{p}(2)}_{=1} = \frac{2}{n(n-1)}$$

CUT(G) — G connected multigraph

1: $G' \leftarrow G$
2: **while** $|V(G')| > 2$ **do**
3: $e \leftarrow$ uniformly distributed random edge in G'
4: $G' \leftarrow G'/e$
5: **return** result of $O(t^4)$ algorithm on G'

→ we can increase the probability of success by being more careful with the first steps → Contract G' with t vertices, then use the randomized $O(t^4)$ algorithm:

→ $\hat{p}_t(G) :=$ Probability, that $\mu(G)$ is returned

→ $\hat{p}_t(n) := \inf_{G=(V,E),|V|=n} \hat{p}_t(G)$, $\hat{p}_t(t) \geq 1 - e^{-1} = \frac{e-1}{e}$

→ $\hat{p}_t(n) \geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdots \frac{t+1}{t+3} \cdot \frac{t}{t+2} \cdot \frac{t-1}{t+1} \cdot \hat{p}_t(t) \geq \underbrace{\frac{t(t-1)}{n(n-1)} \cdot \frac{e-1}{e}}_{\bar{p}_t(n) :=}$

→ $\lambda \frac{1}{\bar{p}_t(n)}$ - time repetition gives error probability $\leq e^{-\lambda}$ and runtime:

$\underbrace{\lambda \frac{n(n-1)}{t(t-1)} \frac{e}{e-1}}_{\#\text{repetitions}} \cdot O(\underbrace{n(n-t)}_{\text{reduction to } t \text{ vertices}} + \underbrace{t^4}_{\text{alg. over } t^2 \text{ edges}}) = O(\lambda(\frac{n^4}{t^2} + n^2 t^2))$, for $t := \sqrt{n}$, success prob. $\geq 1 - e^{-\lambda}$ and runtime $O(\lambda n^3)$

→ If we do this with the $O(n^3)$ algorithm instead of $O(n^4)$, we get a better algorithm and so on... (bootstrapping)

→ If we have an $O(n^c)$ algorithm with success probability $1-e^{-1}$, you get an algorithm with success probability $1-e^{-\lambda}$ and runtime:

$\underbrace{\lambda \frac{n(n-1)}{t(t-1)} \frac{e}{e-1}}_{\#\text{repetitions}} \cdot O(\underbrace{n(n-t)}_{\text{reduction to } t \text{ vertices}} + \underbrace{t^c}_{\text{alg. over } t^2 \text{ edges}}) = O(\lambda(\frac{n^4}{t^2} + n^{2+c-2}))$ → for $t := n^{2/c}$: Success prob. $\geq 1 - e^{-\lambda}$, runtime $O(\lambda n^{4-4/c})$

⇒ runtime goes from $n^4 \to n^3 \to n^{2.666} \to n^{2.5} \to n^{2.4} \ldots$

→ after k-time repetition, we get runtime $O(n^{2+2/k})$, which "goes to" a $O(n^2 \operatorname{polylog}(n))$ algorithm

Algorithm:

KargerStein(G) — G connected multigraph

1: if $|V(G)| \leq 6$ then
2: solve problem in constant time
3: $G' \leftarrow G$, $t := \lceil |V(G)|/\sqrt{2}+1 \rceil$
4: while $|V(G')| > t$ do
5: $e \leftarrow$ uniformly randomly selected edge in G'
6: $G' \leftarrow G'/e$
7: $C_1 :=$ KargerStein(G'), $C_2 :=$ KargerStein(G_2)
8: return the smaller set of C_1, C_2

Smallest Enclosed Disk:

Problem: Given a finite set of points $P \subseteq \mathbb{R}^2$, compute the disk with the smallest radius that encloses P.

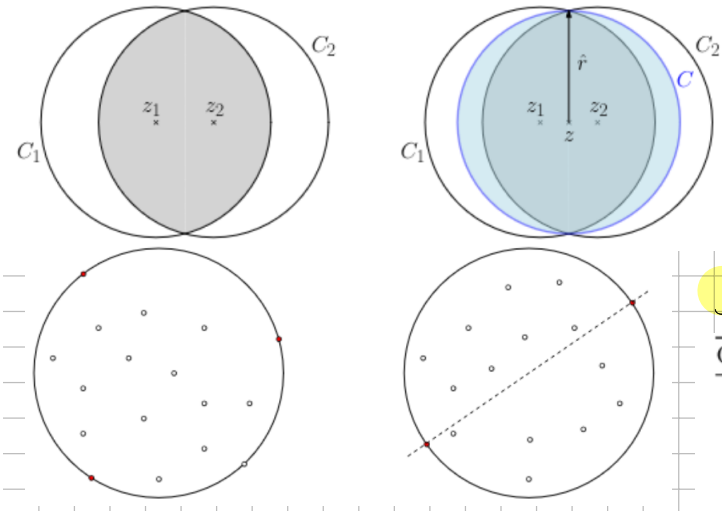
→ We define the closed disk bounded by a disk C as $C^\bullet$

→ C encloses P if $P \subseteq C^\bullet$, points in P can also lie on C.

Lemma: for all finite sets of points $P \subseteq \mathbb{R}^2$ there is a unique smallest enclosing disk $C(P)$.

Lemma: For any set of points $P \subseteq \mathbb{R}^2$, $|P| \geq 3$, there exists a subset $Q \subseteq P$, so that $|Q| = 3$ and $C(Q) = C(P)$

(Q is a certificate for $C(P)$)



Simple algorithms:

COMPLETEENUMERATION(P)

1: **for all** $Q \in \binom{P}{3}$ **do**
2: compute $C(Q)$
3: **if** $P \subseteq C^\bullet(Q)$ **then**
4: **return** $C(Q)$

→ We iterate over $\binom{n}{3}$ sets Q, compute $C(Q)$ in $O(1)$, and validate $P \subseteq C^\bullet(Q)$ in $O(n)$ time

⇒ overall runtime of $O(n^4)$

→ we used the fact that $\forall Q \subseteq P, \text{radius}(C(Q)) \leq \text{radius}(C(P)) \Rightarrow C^*(Q) \subseteq C^*(P)$ does not need to hold

CompleteEnumerationSmart(P)
1: $r \leftarrow 0$
2: **for all** $Q \in \binom{P}{3}$ **do** compute $C(Q)$
3: **if** radius($C(Q)$) $> r$ **then**
4: $C^* \leftarrow C(Q)$; $r \leftarrow$ radius($C(Q)$)
5: **return** C^*

→ $O(n^3)$ runtime, we made use of the following facts:

→ $\forall Q \subseteq P$, $\text{radius}(C(Q)) \leq \text{radius}(C(P))$, $\forall Q \subseteq P$ with $\text{radius}(C(Q)) = \text{radius}(C(P))$: $C(Q) = C(P)$

Randomized_PrimitiveVersion(P)
1: repeat forever
2: choose $Q \subseteq P$ with $|Q| = 3$ randomly, uniformly
3: compute $C(Q)$
4: if $P \subseteq C^*(Q)$ then
5: return $C(Q)$

→ we get the correct Q with probability $\geq 1/\binom{n}{3}$

→ expected attempts until success $\leq \binom{n}{3} \Rightarrow$ expected runtime $O(n^4)$

→ Idea: we find out what points lie outside $C(Q)$ → more useful to know than the points inside of $C(Q)$

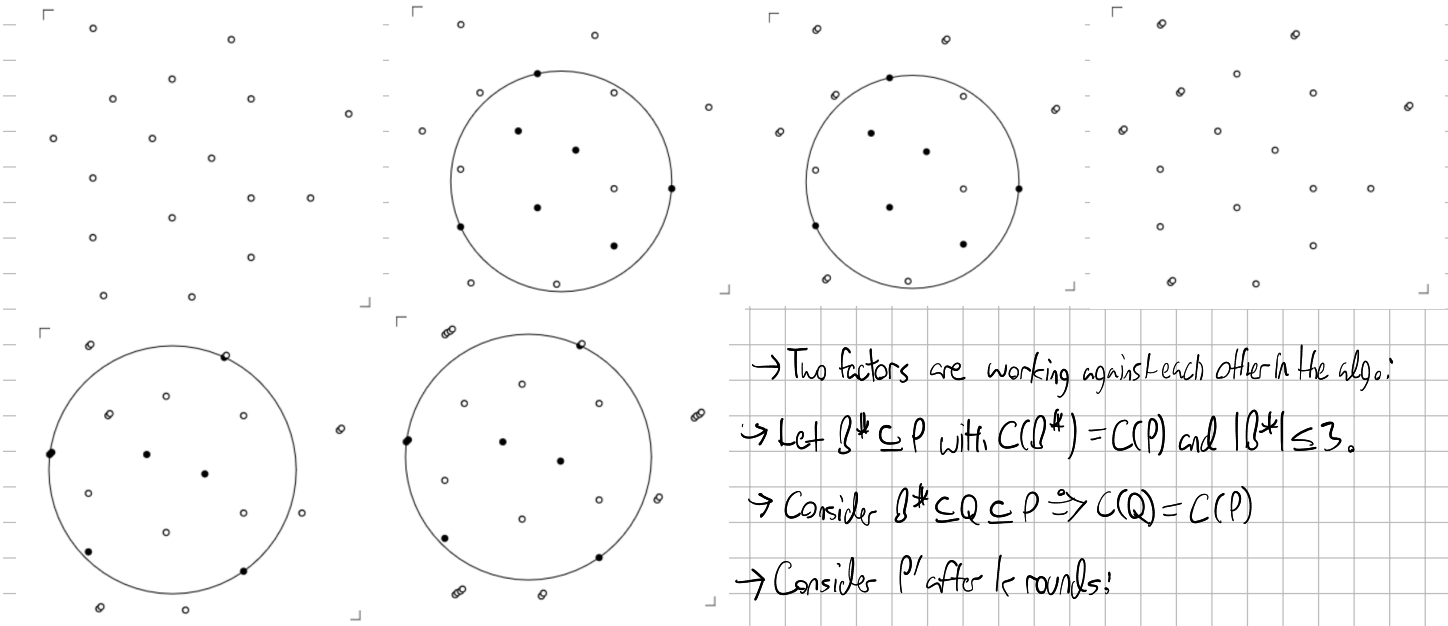
Randomized_CleverVersion(P)
1: $P' \leftarrow P$
2: repeat
3: choose $Q \subseteq P'$ with $|Q| = 11$ randomly, uniformly
4: compute $C(Q)$
5: if $P \subseteq C^*(Q)$ then return $C(Q)$
6: else double all points of P' outside of $C(Q)$

→ We require that the choice of Q is possible in $O(n)$, $n := |P|$

→ we can implement each round in $O(n)$ this way

→ how many rounds until $P \subseteq C^*(Q)$?

Algorithm in action:



→ Two factors are working against each other in the algo.:

→ Let $B^* \subseteq P$ with $C(B^*) = C(P)$ and $|B^*| \leq 3$.

→ Consider $B^* \subseteq Q \subseteq P \Rightarrow C(Q) = C(P)$

→ Consider P' after k rounds:

→ **Case 1:** $|P'|$ grows fast: If $P \not\subseteq C^*(Q)$, there exists a point in B^* outside of $C(Q)$ → some point in B^* must have been doubled $k/3$ times and have multiplicity $\geq 2^{k/3} \Rightarrow |P'| \geq 2^{k/3}$

→ **Case 2:** $|P'|$ does not grow fast: The expected value of $|P'|$ only increases by a factor $\frac{5}{4}$ (to be shown) → $\mathbb{E}[|P'|] \leq \left(\frac{5}{4}\right)^k n$

Sampling Lemma: Let $r, N \in \mathbb{N}$, $r \leq N$, $P' \subseteq \mathbb{R}^2$ a multiset (→ set where elements can appear multiple times), $|P'| = N$. For uniformly random R from $\binom{P'}{r}$:

$$\mathbb{E}[\underbrace{|P' \setminus C(R)|}_{\text{points in } P' \text{ outside of } C(R)}] \leq 3\frac{N-r}{r+1} \leq 3\frac{N}{r+1}$$

Proof: for $s \in P'$, $R, Q \subseteq P'$, define indicators $\text{out}(s,R) := \begin{cases} 1, & s \notin C^*(R) \\ 0 & \text{else} \end{cases}$, $\text{ess}(s,Q) := \begin{cases} 1, & C(Q \setminus \{s\}) \neq C(Q) \\ 0 & \text{else} \end{cases}$

→ $\sum_{s \in P' \setminus R} \text{out}(s,R) = |P' \setminus C^*(R)|$, $\sum_{s \in Q} \text{ess}(s,Q) \leq 3 \Rightarrow \text{out}(s,R) = 1 \Leftrightarrow \text{ess}(s, R \cup \{s\}) = 1$

→ $\mathbb{E}[|P' \setminus C^*(R)|] = \mathbb{E}[\sum_{s \in P' \setminus R} \text{out}(s,R)] = \frac{1}{\binom{N}{r}} \sum_{R \in \binom{P'}{r}} \sum_{s \in P' \setminus R} \text{out}(s,R) = \frac{1}{\binom{N}{r}} \sum_{R \in \binom{P'}{r}} \sum_{s \in P' \setminus R} \text{css}(s, R \cup \{s\})$

$= \frac{1}{\binom{N}{r}} \sum_{Q \in \binom{P'}{r+1}} \underbrace{\sum_{s \in Q} \text{ess}(s,Q)}_{\leq 3} \leq \frac{1}{\binom{N}{r}} \sum_{Q \in \binom{P'}{r+1}} 3 = 3 \cdot \frac{\binom{N}{r+1}}{\binom{N}{r}} = 3\frac{N-r}{r+1}$ ▨

→ how many rounds in the algorithm until $P \subseteq C^*(Q)$?

Amount of points after k rounds:

→ **Sampling Lemma:** $\mathbb{E}[|P' \setminus C^*(R)|] \leq 3\frac{N}{r+1}$ with $N = |P'|$

→ double the points in $P'(C^*(R)) \Rightarrow$ new amount of points in expected value $\leq N + 3\frac{N}{r+1} \leq (1+\frac{3}{r+1})N = \frac{5}{4}N$ for $r = 11$

→ after 0 iterations: $|P'| = n$, after one iteration: $\mathbb{E}[|P'|] \leq \frac{5}{4}n \Rightarrow$ k iterations: $\mathbb{E}[|P'|] \leq (\frac{5}{4})^k n$

Let $T \in \mathbb{N} \cup \{\infty\}$ be the **amount of rounds** of the algorithm and $X_k := |P'|$ for P' after $\min\{T, k\}$ rounds ($X_0 = |P| = n$).

→ $\mathbb{E}[X_k] = \sum_{t=0}^{\infty} \mathbb{E}[X_k | X_{k-1} = t] \cdot \Pr[X_{k-1} = t] \leq \sum_{t=0}^{\infty} (1 + \frac{3}{r+1})t \cdot \Pr[X_{k-1} = t]$

$= (1+\frac{3}{r+1}) \cdot \sum_{t=0}^{\infty} t \cdot \Pr[X_{k-1} = t] = (1+\frac{3}{r+1}) \cdot \mathbb{E}[X_{k-1}]$

→ because $X_0 = n$, $\mathbb{E}[X_k] \leq (1+\frac{3}{r+1})^k \cdot n$

Summarized: Recall that if $T \geq k$, $X_k \geq 2^{k/3}$. We know $\mathbb{E}[X_k] \leq (1+\frac{3}{11+1})^k \cdot n = (\frac{5}{4})^k \cdot n$.

Also: $\mathbb{E}[X_k] = \underbrace{\mathbb{E}[X_k | T \geq k]}_{\geq 2^{k/3}} \cdot \Pr[T \geq k] + \underbrace{\mathbb{E}[X_k | T < k]}_{\geq 0} \cdot \Pr[T < k] \geq 2^{k/3} \cdot \Pr[T \geq k]$

$\Rightarrow 2^{k/3} \cdot \Pr[T \geq k] \leq \mathbb{E}[X_k] \leq (\frac{5}{4})^k \cdot n \Rightarrow \Pr[T \geq k] \leq \underbrace{(\frac{5}{4 \cdot 2^{1/3}})^k}_{\leq 0,993} \cdot n \leq 0.993^k n \leq 1$ (if $k \geq -\log_{0,993} n$)

Theorem: The algorithm computes the smallest enclosing disk in expected time $O(n \log n)$.

Proof: Runtime $= O(nT)$. Set $k_0 := \lfloor \log_{0,993} n \rfloor$.

$\mathbb{E}[T] = \sum_{k \geq 1} \Pr[T \geq k] \leq \sum_{k=1}^{k_0} 1 + \sum_{k > k_0} 0,993^k n = \sum_{k=1}^{k_0} 1 + \sum_{k'=0} 0,993^{k'} \cdot \underbrace{0.993^{k_0+1}}_{\leq 1}$

$= k_0 + O(1) = O(\log n)$